

How to Use This Coding Guide

The SP/DSE coding round presents 3 problems (Easy, Medium, Hard) with an emphasis on Dynamic Programming, Greedy techniques, arrays, strings and two-pointer patterns.

Specialist Programmer problems trend towards competitive-programming difficulty, so master the patterns below rather than memorising code.

Every Python solution here has been executed and verified against its sample output.

Coding Problems

Problem 1: Second Largest Element in an Array

Difficulty: Easy | Pattern: Array Traversal

Problem Statement

Given an array of N integers, find the second largest element. If it does not exist, print -1. Duplicates of the largest value are ignored when finding the second largest.

Input Format

First line: N. Second line: N space-separated integers.

Output Format

A single integer: the second largest element, or -1.

Constraints

$1 \leq N \leq 10^5$, $-10^9 \leq a[i] \leq 10^9$

Sample Input

```
6
12 35 1 10 34 1
```

Sample Output

```
34
```

Python Solution

```
def second_largest(arr):
    largest = second = float('-inf')
    for x in arr:
        if x > largest:
            second = largest
            largest = x
        elif largest > x > second:
            second = x
    return second if second != float('-inf') else -1

arr = [12, 35, 1, 10, 34, 1]
print(second_largest(arr))
```

Step-by-Step Explanation

Track the largest and second largest in a single pass. When a new maximum appears, the old maximum becomes the second largest. A value strictly between the current second and largest updates the second largest. This avoids sorting and runs in linear time.

Dry Run

```
arr = [12,35,1,10,34,1]
12 -> largest=12, second=-inf
35 -> largest=35, second=12
1  -> no change
10 -> second=12 (10<12) no change
34 -> 35>34>12 -> second=34
1  -> no change
Result = 34
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 2: Maximum Subarray Sum (Kadane's Algorithm)

Difficulty: **Easy-Medium** | Pattern: **Dynamic Programming / Greedy**

Problem Statement

Given an array of N integers (which may be negative), find the maximum sum obtainable from any contiguous non-empty subarray.

Input Format

First line: N. Second line: N space-separated integers.

Output Format

A single integer: the maximum contiguous subarray sum.

Constraints

$1 \leq N \leq 10^5$, $-10^4 \leq a[i] \leq 10^4$

Sample Input

```
9
-2 1 -3 4 -1 2 1 -5 4
```

Sample Output

```
6
```

Python Solution

```
def max_subarray_sum(arr):
    best = current = arr[0]
    for x in arr[1:]:
        current = max(x, current + x)
        best = max(best, current)
    return best

arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print(max_subarray_sum(arr))
```

Step-by-Step Explanation

Kadane's algorithm keeps a running sum 'current'. At each element we decide whether to extend the previous subarray (current + x) or start fresh at x. 'best' records the maximum seen so far. The subarray

[4,-1,2,1] gives the maximum sum 6.

Dry Run

```
current=best=-2
1 -> current=max(1, -1)=1, best=1
-3-> current=max(-3, -2)=-2, best=1
4 -> current=max(4, 2)=4, best=4
-1-> current=3, best=4
2 -> current=5, best=5
1 -> current=6, best=6
-5-> current=1, best=6
4 -> current=5, best=6
Result = 6
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 3: Move All Zeroes to the End

Difficulty: **Easy** | Pattern: **Two Pointers (in-place)**

Problem Statement

Given an array of N integers, move all zeroes to the end while maintaining the relative order of the non-zero elements. Do it in-place.

Input Format

First line: N. Second line: N space-separated integers.

Output Format

The modified array, space-separated on one line.

Constraints

$1 \leq N \leq 10^5$, $-10^9 \leq a[i] \leq 10^9$

Sample Input

```
7
0 1 0 3 12 0 5
```

Sample Output

```
1 3 12 5 0 0 0
```

Python Solution

```
def move_zeroes(arr):
    insert_pos = 0
    for x in arr:
        if x != 0:
            arr[insert_pos] = x
            insert_pos += 1
    while insert_pos < len(arr):
        arr[insert_pos] = 0
        insert_pos += 1
    return arr

arr = [0, 1, 0, 3, 12, 0, 5]
print(' '.join(map(str, move_zeroes(arr))))
```

Step-by-Step Explanation

Use a write pointer 'insert_pos'. Copy every non-zero value forward, keeping order. After processing, fill the remaining positions with zeroes. This is a single pass plus a fill pass, all in-place.

Dry Run

```
Non-zero pass writes: 1,3,12,5 at indices 0..3
insert_pos=4, fill zeroes at 4,5,6
Result = 1 3 12 5 0 0 0
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 4: Coin Change - Minimum Number of Coins

Difficulty: **Medium** | Pattern: **Dynamic Programming (unbounded knapsack)**

Problem Statement

Given coin denominations and a target amount, find the minimum number of coins needed to make the amount. Each coin can be used unlimited times. If it cannot be formed, print -1.

Input Format

Line 1: number of coin types and the amount. Line 2: the coin denominations.

Output Format

Minimum number of coins, or -1 if impossible.

Constraints

$1 \leq \text{coins} \leq 12$, $1 \leq \text{amount} \leq 10^4$, $1 \leq \text{coin}[i] \leq 10^4$

Sample Input

```
3 11
1 2 5
```

Sample Output

```
3
```

Python Solution

```
def coin_change(coins, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1

coins = [1, 2, 5]
print(coin_change(coins, 11))
```

Step-by-Step Explanation

dp[a] holds the fewest coins to make amount a. Base case dp[0]=0. For every amount a, try each coin c: if c fits, dp[a] can be improved to dp[a-c]+1. The answer is dp[amount]. 11 = 5 + 5 + 1 uses 3 coins.

Dry Run

```
dp[0]=0
dp[1]=1 (1)
```

```
dp[2]=1 (2)
dp[5]=1 (5)
dp[10]=2 (5+5)
dp[11]=dp[6]+1=3 (5+5+1)
Result = 3
```

Time Complexity: **O(amount * number_of_coins)** | Space Complexity: **O(amount)**

Problem 5: Longest Increasing Subsequence

Difficulty: **Medium** | Pattern: **Dynamic Programming**

Problem Statement

Given an array of N integers, find the length of the longest strictly increasing subsequence (elements need not be contiguous).

Input Format

First line: N. Second line: N space-separated integers.

Output Format

A single integer: the length of the LIS.

Constraints

$1 \leq N \leq 2500$, $-10^9 \leq a[i] \leq 10^9$

Sample Input

```
8
10 9 2 5 3 7 101 18
```

Sample Output

```
4
```

Python Solution

```
def length_of_lis(arr):
    n = len(arr)
    dp = [1] * n
    for i in range(n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

arr = [10, 9, 2, 5, 3, 7, 101, 18]
print(length_of_lis(arr))
```

Step-by-Step Explanation

$dp[i]$ is the length of the longest increasing subsequence ending at index i . For each i , look back at all $j < i$; if $arr[j] < arr[i]$, the subsequence ending at j can be extended. The answer is the maximum over dp . One LIS is $[2, 3, 7, 18]$ or $[2, 5, 7, 101]$, length 4.

Dry Run

```
dp starts all 1
i=3(5): arr[2]=2<5 -> dp=2
i=4(3): 2<3 -> dp=2
i=5(7): 2,5,3<7 -> dp=3
```

```
i=6(101): extends 7 -> dp=4
i=7(18): extends 7 -> dp=4
max(dp)=4
```

Time Complexity: $O(N^2)$ | Space Complexity: $O(N)$

Problem 6: 0/1 Knapsack - Maximum Value

Difficulty: *Medium-Hard* | Pattern: *Dynamic Programming*

Problem Statement

Given N items each with a weight and a value, and a knapsack of capacity W, choose a subset of items (each at most once) to maximise the total value without exceeding the capacity.

Input Format

Line 1: N and W. Line 2: N weights. Line 3: N values.

Output Format

A single integer: the maximum achievable value.

Constraints

$1 \leq N \leq 100$, $1 \leq W \leq 1000$, $1 \leq wt, val \leq 1000$

Sample Input

```
3 50
10 20 30
60 100 120
```

Sample Output

```
220
```

Python Solution

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [0] * (capacity + 1)
    for i in range(n):
        for w in range(capacity, weights[i] - 1, -1):
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
    return dp[capacity]

weights = [10, 20, 30]
values = [60, 100, 120]
print(knapsack(weights, values, 50))
```

Step-by-Step Explanation

$dp[w]$ is the best value achievable with capacity w. We process items one at a time and iterate capacity from high to low so each item is used at most once. Taking items 2 and 3 (weights $20+30=50$) gives value $100+120=220$.

Dry Run

```
After item1(10,60): dp[10..]=60
After item2(20,100): dp[30]=160
After item3(30,120): dp[50]=max(160, dp[20]+120=220)=220
Result = 220
```

Time Complexity: $O(N * W)$ | Space Complexity: $O(W)$

Problem 7: Longest Substring Without Repeating Characters

Difficulty: *Medium* | Pattern: *Sliding Window + Hash Map*

Problem Statement

Given a string, find the length of the longest substring that contains no repeating characters.

Input Format

A single line containing the string S.

Output Format

A single integer: the length of the longest such substring.

Constraints

$0 \leq |S| \leq 10^5$, S consists of printable ASCII characters.

Sample Input

abcabcbb

Sample Output

3

Python Solution

```
def longest_unique_substring(s):
    last_seen = {}
    start = 0
    best = 0
    for i, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= start:
            start = last_seen[ch] + 1
        last_seen[ch] = i
        best = max(best, i - start + 1)
    return best

print(longest_unique_substring('abcabcbb'))
```

Step-by-Step Explanation

Maintain a sliding window [start, i]. 'last_seen' stores the most recent index of each character. When a repeat inside the window is found, move 'start' just past the previous occurrence. Track the longest window. For 'abcabcbb' the answer is 'abc' with length 3.

Dry Run

```
i=0 a: window 'a' len1
i=1 b: 'ab' len2
i=2 c: 'abc' len3 (best=3)
i=3 a: repeat -> start=1, 'bca' len3
... max stays 3
Result = 3
```

Time Complexity: $O(N)$ | Space Complexity: $O(\min(N, \text{alphabet}))$

Problem 8: Trapping Rain Water

Difficulty: **Hard** | Pattern: **Two Pointers**

Problem Statement

Given N non-negative integers representing an elevation map where the width of each bar is 1, compute how much water can be trapped after raining.

Input Format

First line: N. Second line: N space-separated heights.

Output Format

A single integer: the total units of trapped water.

Constraints

$0 \leq N \leq 10^5$, $0 \leq \text{height}[i] \leq 10^4$

Sample Input

```
12
0 1 0 2 1 0 1 3 2 1 2 1
```

Sample Output

```
6
```

Python Solution

```
def trap(height):
    left, right = 0, len(height) - 1
    left_max = right_max = 0
    water = 0
    while left < right:
        if height[left] < height[right]:
            left_max = max(left_max, height[left])
            water += left_max - height[left]
            left += 1
        else:
            right_max = max(right_max, height[right])
            water += right_max - height[right]
            right -= 1
    return water

h = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]
print(trap(h))
```

Step-by-Step Explanation

Two pointers move inward. Water above a bar is limited by the shorter of the tallest bars to its left and right. We advance the side with the smaller height, because that side determines the trapped water there, accumulating (running max - current height).

Dry Run

The classic input traps water in the dips between the tall bars (heights 2 and 3). Summing per-

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 9: Edit Distance (Levenshtein)

Difficulty: **Hard** | Pattern: **Dynamic Programming (2D)**

Problem Statement

Given two strings word1 and word2, find the minimum number of single-character operations (insert, delete, replace) required to convert word1 into word2.

Input Format

Line 1: word1. Line 2: word2.

Output Format

A single integer: the minimum edit distance.

Constraints

$0 \leq |word1|, |word2| \leq 500$

Sample Input

```
horse
ros
```

Sample Output

```
3
```

Python Solution

```
def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j],
                                    dp[i][j - 1],
                                    dp[i - 1][j - 1])
    return dp[m][n]

print(edit_distance('horse', 'ros'))
```

Step-by-Step Explanation

$dp[i][j]$ is the edit distance between the first i characters of a and first j of b . If characters match, no new operation is needed. Otherwise take 1 plus the best of delete, insert, or replace. horse $\rightarrow$ rorse $\rightarrow$ rose $\rightarrow$ ros needs 3 operations.

Dry Run

```
Base row/col are 0..n and 0..m.
Filling the table, dp[5][3] resolves to 3 via replace h->r, replace r->o (kept), delete e, dele
Final dp[5][3] = 3
```

Time Complexity: **$O(m * n)$** | Space Complexity: **$O(m * n)$**



Coding Problems (DSA)

Applied Role(s):

Digital Specialist Engineer (Trainee) | Specialist Programmer L1 / L2 / L3 (Trainee)

Online Assessment Preparation Material

Version 1.0 • Last Updated: July 2026

Compiled from verified previous-year patterns, community-reported questions and research-backed company-style practice.

Table of Contents

How to Use This Coding Guide	1
Coding Problems	1